



Ansible Controller Server Setup & Configuration

Overview

This document details the complete transformation of a standard Ubuntu VM into an enterprise-grade Ansible management server. The implementation establishes professional automation infrastructure with dual-account architecture, static networking, cross-platform management capabilities, and comprehensive Windows integration.

Server Transformation: - **Original:** srv1-ubuntu-vm (DHCP: 192.168.10.50) - **Final:** ansible-mgmt-01 (Static: 192.168.10.2)

Key Capabilities: - Cross-platform automation (Linux + Windows) - SSH key-based authentication with custom naming - Static IP with cloud-init override protection - WinRM integration for Windows management - Enterprise-grade service account architecture

Network Configuration

Management VLAN Infrastructure

Component	Details
VLAN ID	10 (Management)
Subnet	192.168.10.0/24
Gateway	192.168.10.1 (pfSense)
Ansible Controller IP	192.168.10.2 (Static)
Interface	ens37

Current Management VLAN Systems

System	IP Address	Purpose	OS	Status
pfSense	192.168.10.1	Gateway/Firewall	pfSense	Active
ansible-mgmt-01	192.168.10.2	Ansible Controller	Ubuntu 24.04	Active
Windows Host	192.168.10.3	Lab Workstation	Windows 11	Active

System	IP Address	Purpose	OS	Status
TCM Ubuntu	192.168.10.4	Lab System	Ubuntu 24.04	Active
Windows Server	192.168.10.5	Domain Controller	Windows Server 2022	Active
vault-server	192.168.10.6	Secrets Management	Ubuntu 25.04	Active

Phase 1: Service Account Architecture

Dual-Account Model Rationale

Why Two Accounts?

In enterprise environments, service accounts provide critical separation:

- **Separation of Duties:** Human actions vs automated actions are distinct
- **Security Isolation:** Different SSH keys for different purposes
- **Clear Audit Trails:** Log entries distinguish between admin and automation
- **Least Privilege:** Service accounts have only necessary permissions

Account Structure

```
Your Laptop (Admin)
  ↓ SSH as 'nantwi'
ansible-mgmt-01 (Human administrative work)
  ↓ sudo su - svc-ansible
svc-ansible account (Automation operations)
  ↓ SSH with ansible-automation-key
Managed Hosts (Linux + Windows systems)
```

Account Implementation

Step 1: Create Service Account

```
# Create the svc-ansible user
sudo adduser svc-ansible
```

Interactive prompts: - Password: [Set strong password] - Full Name: Ansible Service Account - Other fields: [Press Enter to skip]

Step 2: Grant Sudo Access

```
# Add to sudo group
sudo usermod -aG sudo svc-ansible
```

```
# Verify group membership
groups svc-ansible
```

Expected output:

```
svc-ansible : svc-ansible sudo
```

Step 3: Configure Passwordless Sudo

```
# Create sudoers file for automation
echo "svc-ansible ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/svc-ansible

# Set secure permissions (critical!)
sudo chmod 0440 /etc/sudoers.d/svc-ansible

# Verify configuration
sudo cat /etc/sudoers.d/svc-ansible
```

Why passwordless sudo? - Ansible automation requires non-interactive privilege escalation - Service accounts don't have human operators to enter passwords - Still secure: requires initial authentication to switch to svc-ansible

Step 4: Test Service Account

```
# Switch to service account
sudo su - svc-ansible

# Verify identity
whoami
# Output: svc-ansible

# Test passwordless sudo
sudo whoami
# Output: root (no password prompt!)

# Exit back to admin account
exit
```

Phase 2: Server Hostname Configuration

Step 1: Set New Hostname

```
# Change hostname
sudo hostnamectl set-hostname ansible-mgmt-01
```

```
# Verify change
hostnamectl
```

Expected output:

```
Static hostname: ansible-mgmt-01
Icon name: computer-vm
Chassis: vm
```

Step 2: Update /etc/hosts

```
# Backup original file
sudo cp /etc/hosts /etc/hosts.backup

# View current content
cat /etc/hosts
```

Original content:

```
127.0.0.1 localhost
127.0.1.1 srv1-ubuntu-vm
```

```
# Replace old hostname
sudo sed -i 's/srv1-ubuntu-vm/ansible-mgmt-01/g' /etc/hosts

# Verify changes
cat /etc/hosts
```

Updated content:

```
127.0.0.1 localhost
127.0.1.1 ansible-mgmt-01
```

Step 3: Configure Cloud-Init Protection

Check if cloud-init is present:

```
which cloud-init
# Output: /usr/bin/cloud-init
```

Prevent cloud-init from overwriting hostname:

```
# Method 1: Set preserve_hostname flag
sudo sed -i 's/preserve_hostname: false/preserve_hostname: true/g' /etc/cloud/cloud.cfg

# Verify
```

```
sudo grep "preserve_hostname" /etc/cloud/cloud.cfg
# Should show only ONE line: preserve_hostname: true
```

If duplicate entries exist:

```
# Edit manually
sudo nano /etc/cloud/cloud.cfg
# Find and fix: ensure only ONE "preserve_hostname: true" line exists
```

Method 2: Create override configuration:

```
# Create explicit hostname protection
sudo bash -c 'cat > /etc/cloud/cloud.cfg.d/99-disable-hostname.cfg << EOF
#cloud-config
hostname: ansible-mgmt-01
preserve_hostname: true
manage_etc_hosts: false
EOF'

# Verify creation
cat /etc/cloud/cloud.cfg.d/99-disable-hostname.cfg
```

Disable cloud-init network management (important for static IP):

```
# Prevent cloud-init from managing network
sudo bash -c 'cat > /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg << EOF
network: {config: disabled}
EOF'

# Verify
cat /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
```

Phase 3: SSH Key Configuration

Custom SSH Key Generation

Standard practice uses `id_ed25519`, but custom naming provides better clarity in enterprise environments.

Step 1: Switch to Service Account

```
# Switch to svc-ansible
sudo su - svc-ansible

# Verify identity
```

```
whoami
# Output: svc-ansible
```

Step 2: Generate SSH Key Pair

```
# Generate key with descriptive name
ssh-keygen -t ed25519 -C "svc-ansible@ansible-mgmt-01" -f ~/.ssh/ansible-automation-key
```

Prompts: - Enter passphrase: **[Press Enter - no passphrase for automation]** - Enter same passphrase again: **[Press Enter]**

Why no passphrase? - Automation requires non-interactive SSH - Key is protected by file system permissions - Access to key requires access to svc-ansible account

Expected output:

```
Generating public/private ed25519 key pair.
Your identification has been saved in /home/svc-ansible/.ssh/ansible-automation-key
Your public key has been saved in /home/svc-ansible/.ssh/ansible-automation-key.pub
The key fingerprint is:
SHA256:xxxx... svc-ansible@ansible-mgmt-01
```

Step 3: Verify Key Creation

```
# List SSH directory
ls -la ~/.ssh/
```

Expected output:

```
drwx----- 2 svc-ansible svc-ansible 4096 Jan 12 10:30 .
drwxr-x--- 3 svc-ansible svc-ansible 4096 Jan 12 10:25 ..
-rw----- 1 svc-ansible svc-ansible 411 Jan 12 10:30 ansible-automation-key
-rw-r--r-- 1 svc-ansible svc-ansible 103 Jan 12 10:30 ansible-automation-key.pub
```

Note the permissions: - Private key: `-rw-----` (600) - owner read/write only - Public key: `-rw-r--r--` (644) - readable by all

Step 4: Configure SSH to Use Custom Key

```
# Create SSH config file
nano ~/.ssh/config
```

Add configuration:

```
# Ansible managed hosts default key
Host *
```

```
IdentityFile ~/.ssh/ansible-automation-key
IdentitiesOnly yes
StrictHostKeyChecking accept-new
```

Configuration explanation: - `Host *`: Applies to all SSH connections - `IdentityFile`: Always use custom-named key - `IdentitiesOnly yes`: Don't try default key names - `StrictHostKeyChecking accept-new`: Auto-accept new hosts (safe for homelab)

Set proper permissions:

```
chmod 600 ~/.ssh/config

# Verify
ls -la ~/.ssh/config
# Output: -rw----- 1 svc-ansible svc-ansible ...
```

Step 5: Display Public Key

```
# View public key (needed for managed hosts)
cat ~/.ssh/ansible-automation-key.pub
```

Example output:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGx... svc-ansible@ansible-mgmt-01
```

Save this public key - you'll need it to configure managed hosts!

Step 6: Exit Service Account

```
# Return to admin account
exit
```

Phase 4: Static IP Configuration

Current State

- Interface: ens37
- Current IP: 192.168.10.50 (DHCP)
- Target IP: 192.168.10.2 (Static)

Why Static IP?

For infrastructure servers: - **Predictable addressing:** Always accessible at same IP - **Firewall rules:** Can create stable ACLs - **Documentation:** IP never changes - **DNS/Hosts files:** Reliable name resolution

Netplan Configuration

Step 1: Check Current Network Config

```
# List netplan configuration files
ls -la /etc/netplan/
```

Typical output:

```
-rw----- 1 root root 64 Jan 10 18:30 50-cloud-init.yaml
```

Step 2: Backup Existing Configuration

```
# Create backup
sudo cp /etc/netplan/50-cloud-init.yaml /etc/netplan/50-cloud-init.yaml.bak

# Verify backup
ls -la /etc/netplan/
```

Step 3: Remove Cloud-Init Network File

```
# Remove cloud-init managed network config
sudo rm /etc/netplan/50-cloud-init.yaml

# Verify removal
ls -la /etc/netplan/

# Should only show: 50-cloud-init.yaml.bak
```

Step 4: Create Static IP Configuration

```
# Create new netplan configuration
sudo nano /etc/netplan/01-netcfg.yaml
```

Add this configuration:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens37:
      dhcp4: no
      addresses:
        - 192.168.10.2/24
      routes:
        - to: default
          via: 192.168.10.1
```



```
nameservers:
  addresses:
    - 8.8.8.8
    - 8.8.4.4
```

Configuration breakdown: - `version: 2` : Netplan format version - `renderer: networkd` : Use systemd-networkd backend - `ens37` : Your network interface name - `dhcp4: no` : Disable DHCP - `addresses` : Static IP with /24 subnet mask - `routes` : Default gateway through pfSense - `nameservers` : Google DNS servers

Save and exit: `Ctrl+O` , `Enter` , `Ctrl+X`

Step 5: Set Proper Permissions

```
# Netplan requires strict permissions
sudo chmod 600 /etc/netplan/01-netcfg.yaml

# Verify
ls -la /etc/netplan/
# Output: -rw----- 1 root root ... 01-netcfg.yaml
```

Step 6: Test Configuration (IMPORTANT!)

```
# Test configuration with automatic rollback
sudo netplan try
```

What happens: - Config applies temporarily for 120 seconds - You have 2 minutes to verify connectivity - If you don't press Enter, it auto-reverts (safety feature!)

During the 120-second window:

Open a NEW SSH session from your laptop:

```
# From your laptop
ssh nantwi@192.168.10.2
```

In the original session, test connectivity:

```
# Test gateway
ping -c 4 192.168.10.1

# Test DNS
ping -c 4 8.8.8.8

# Test name resolution
ping -c 4 google.com
```

If all tests pass: - Go back to terminal running `netplan try` - Press `Enter` to accept changes

If tests fail: - Wait 120 seconds - Config automatically reverts to backup - You still have SSH access at old IP

Step 7: Apply Configuration Permanently

```
# Apply the configuration
sudo netplan apply

# Verify new IP address
ip addr show ens37
```

Expected output:

```
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4e:37:96 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.2/24 brd 192.168.10.255 scope global ens37
        valid_lft forever preferred_lft forever
```

Key indicators: - ☒ IP changed from 192.168.10.50 to 192.168.10.2 - ☒ No "dynamic" label (static assignment) - ☒ Shows "scope global" instead of "scope global dynamic"

Step 8: Verify Routing

```
# Check routing table
ip route show
```

Expected output:

```
default via 192.168.10.1 dev ens37
192.168.10.0/24 dev ens37 proto kernel scope link src 192.168.10.2
```

Step 9: Test Complete Connectivity

```
# Test gateway
ping -c 4 192.168.10.1

# Test internet
ping -c 4 8.8.8.8

# Test DNS resolution
ping -c 4 google.com

# Test other lab hosts
ping -c 4 192.168.10.3
ping -c 4 192.168.10.5
```

Phase 5: Post-Configuration Reboot & Verification

Reboot Server

```
# Reboot to ensure all changes persist
sudo reboot
```

SSH session will disconnect - this is expected!

Post-Reboot Verification

Wait 30-60 seconds, then reconnect from your laptop:

```
# SSH with NEW IP address
ssh nantwi@192.168.10.2
```

Verify all changes:

```
# Check hostname
hostname
# Expected: ansible-mgmt-01

# Verify prompt shows new hostname
# Expected: nantwi@ansible-mgmt-01:~$

# Check IP address
ip addr show ens37 | grep "inet "
# Expected: inet 192.168.10.2/24

# Verify routing
ip route show | grep default
# Expected: default via 192.168.10.1 dev ens37

# Test service account
sudo su - svc-ansible
whoami
# Expected: svc-ansible

# Check SSH keys exist
ls -la ~/.ssh/
# Should show: ansible-automation-key and ansible-automation-key.pub

# Exit service account
exit
```

Phase 6: Windows Integration

Windows Host WinRM Configuration

Windows hosts require WinRM (Windows Remote Management) for Ansible management.

On Windows Host (192.168.10.3)

Open PowerShell as Administrator and run:

```
# Enable WinRM service
winrm quickconfig

# Answer 'y' to both prompts:
# - Start WinRM service and set to delayed auto start
# - Enable WinRM firewall exception
```

Expected output:

```
WinRM has been updated to receive requests.
WinRM service type changed successfully.
WinRM service started.
WinRM has been updated for remote management.
WinRM firewall exception enabled.
```

Enable Windows Firewall rule:

```
# Enable the specific WinRM rule
Enable-NetFirewallRule -DisplayName "Windows Remote Management (HTTP-In)"
```

Create ICMP firewall rule (for ping):

```
# Allow ping (ICMP Echo Request)
New-NetFirewallRule -DisplayName "Allow ICMPv4-In" -Protocol ICMPv4 -IcmpType 8 -Enabled True -Act
```



Verify WinRM configuration:

```
# Check WinRM service status
Get-Service WinRM

# Check WinRM listeners
winrm enumerate winrm/config/listener

# Test WinRM locally
Test-WSMan localhost
```

```
# Verify firewall rules
Get-NetFirewallRule -DisplayName "*WinRM*" | Select-Object DisplayName, Enabled, Direction, Action
```

From Ansible Server

Install WinRM Python library:

```
# Install pywinrm
sudo apt update
sudo apt install python3-winrm -y

# Verify installation
python3 -c "import winrm; print('WinRM support installed')"
```

Test port connectivity:

```
# Install nmap if not present
sudo apt install nmap -y

# Test WinRM port
sudo nmap -p 5985 192.168.10.3
```

Expected output:

PORT	STATE	SERVICE
5985/tcp	open	wsman

Test ping:

```
# Test ICMP
ping -c 4 192.168.10.3
```

Expected output:

```
64 bytes from 192.168.10.3: icmp_seq=1 ttl=128 time=X ms
...
4 packets transmitted, 4 received, 0% packet loss
```

Note: TTL=128 confirms Windows host

Phase 7: Windows Server 2022 RDP Configuration

Cross-VLAN RDP Access Setup

Objective: Enable RDP access from ansible-mgmt-01 (192.168.10.2) to Windows Server (192.168.10.5)

On Windows Server 2022 (192.168.10.5)

Step 1: Enable Remote Desktop

```
# Enable Remote Desktop via PowerShell (as Administrator)
Set-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server' -name "fDenyTSConn

# Enable RDP firewall rule
Enable-NetFirewallRule -DisplayGroup "Remote Desktop"

# Verify RDP is enabled
Get-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server' -name "fDenyTSConn
```

Expected output:

```
fDenyTSConnections : 0
```

Step 2: Verify RDP Service

```
# Check Remote Desktop Services
Get-Service TermService

# Should show: Status = Running
```

On pfSense Firewall

Step 3: Configure Firewall Rule (if needed)

Navigate to: **Firewall > Rules > Management VLAN (VLAN 10)**

Create rule: - **Action:** Pass - **Protocol:** TCP - **Source:** 192.168.10.2 (ansible-mgmt-01) - **Destination:** 192.168.10.5 (Windows Server) - **Destination Port:** 3389 (RDP) - **Description:** "Allow RDP from Ansible controller to Windows Server"

Apply changes

From Ansible Controller

Step 4: Test RDP Connectivity

```
# Test RDP port from ansible-mgmt-01
nc -zv 192.168.10.5 3389
```

Expected output:

```
Connection to 192.168.10.5 3389 port [tcp/ms-wbt-server] succeeded!
```

Step 5: RDP Connection Test

From your laptop (which can access ansible-mgmt-01):

```
# If you have RDP client on Linux
xfreerdp /v:192.168.10.5 /u:Administrator

# On Windows laptop, use Remote Desktop Connection:
# Computer: 192.168.10.5
# Username: Administrator
```

Note: Ensure you're connected to the lab network (directly or via Tailscale)

Verification

```
# From ansible-mgmt-01, verify connectivity
ping -c 4 192.168.10.5

# Test RDP port
nc -zv 192.168.10.5 3389

# Test from ansible service account
sudo su - svc-ansible
nc -zv 192.168.10.5 3389
exit
```

Phase 8: Ansible Configuration

Inventory Setup

Create Ansible Hosts File

```
# Edit Ansible inventory
sudo nano /etc/ansible/hosts
```

Add configuration:

```
# Ansible Management Server (self-management)
[controller]
ansible-mgmt-01 ansible_host=192.168.10.2 ansible_connection=local

# Linux Systems
[linux]
192.168.10.4 # TCM-Ubuntu

# Windows Systems
```

```
[windows]
192.168.10.3 # Windows Workstation
192.168.10.5 # Windows Server 2022

# Combined groups
[all_systems:children]
controller
linux
windows
```

Group Variables

Create Group Variables Directory

```
# Create directory structure
sudo mkdir -p /etc/ansible/group_vars

# Set ownership
sudo chown -R svc-ansible:svc-ansible /etc/ansible/group_vars
```

Configure Windows Connection Settings

```
# Create Windows group variables
sudo nano /etc/ansible/group_vars/windows.yml
```

Add configuration:

```
---
# Windows connection settings
ansible_connection: winrm
ansible_winrm_transport: basic
ansible_winrm_server_cert_validation: ignore
ansible_user: Administrator
ansible_password: "YourWindowsPassword"
ansible_port: 5985
```

Important: Replace `YourWindowsPassword` with actual password!

Set secure permissions:

```
sudo chmod 600 /etc/ansible/group_vars/windows.yml
sudo chown svc-ansible:svc-ansible /etc/ansible/group_vars/windows.yml
```

Configure Linux Connection Settings (Optional)

```
# Create Linux group variables
```



```
sudo nano /etc/ansible/group_vars/linux.yml
```

Add configuration:

```
---  
# Linux connection settings  
ansible_connection: ssh  
ansible_user: svc-ansible  
ansible_ssh_private_key_file: ~/.ssh/ansible-automation-key
```

Ansible Configuration File

```
# Edit main Ansible config  
sudo nano /etc/ansible/ansible.cfg
```

Update/add these settings:

```
[defaults]  
inventory = /etc/ansible/hosts  
private_key_file = ~/.ssh/ansible-automation-key  
remote_user = svc-ansible  
host_key_checking = False  
timeout = 30  
gathering = smart  
fact_caching = memory  
  
[privilege_escalation]  
become = True  
become_method = sudo  
become_user = root  
become_ask_pass = False
```

Testing and Verification

Test Ansible Connectivity

Switch to Service Account

```
# Switch to svc-ansible for all Ansible operations  
sudo su - svc-ansible
```

Test Windows Connectivity

```
# Test Windows host with win_ping
ansible windows -m win_ping
```

Expected output:

```
192.168.10.3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
192.168.10.5 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Test Linux Connectivity

```
# Test Linux hosts with ping module
ansible linux -m ping
```

Expected output:

```
192.168.10.4 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

Test All Systems

```
# Test platform-specific connectivity
ansible linux,controller -m ping
ansible windows -m win_ping
```

Get System Information

Windows System Info

```
# Get Windows hostname
ansible windows -m win_shell -a "hostname"

# Get Windows version
ansible windows -m win_shell -a "systeminfo | findstr /B /C:\"OS Name\" /C:\"OS Version\""
```

```
# Get IP configuration
ansible windows -m win_shell -a "ipconfig | findstr IPv4"
```

Linux System Info

```
# Get Linux hostname
ansible linux -m command -a "hostname"

# Get OS information
ansible linux -m command -a "cat /etc/os-release"

# Get IP address
ansible linux -m command -a "ip addr show | grep 'inet '"
```

Configuration Files Reference

Complete Netplan Configuration

File: `/etc/netplan/01-netcfg.yaml`

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens37:
      dhcp4: no
      addresses:
        - 192.168.10.2/24
      routes:
        - to: default
          via: 192.168.10.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

SSH Configuration for Service Account

File: `/home/svc-ansible/.ssh/config`

```
# Ansible managed hosts default key
Host *
  IdentityFile ~/.ssh/ansible-automation-key
  IdentitiesOnly yes
  StrictHostKeyChecking accept-new
```

Cloud-Init Hostname Protection

File: `/etc/cloud/cloud.cfg.d/99-disable-hostname.cfg`

```
#cloud-config
hostname: ansible-mgmt-01
preserve_hostname: true
manage_etc_hosts: false
```

Cloud-Init Network Disable

File: `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg`

```
network: {config: disabled}
```

Sudoers Configuration

File: `/etc/sudoers.d/svc-ansible`

```
svc-ansible ALL=(ALL) NOPASSWD:ALL
```

Troubleshooting

Network Connectivity Issues

Problem: Cannot SSH to New IP

Symptoms: - SSH connection timeout - Connection refused

Solutions:

1. **Connect via VMware console** (direct keyboard/mouse)
2. **Verify IP configuration:** `bash ip addr show ens37 ip route show`
3. **Restore backup if needed:** `bash sudo cp /etc/netplan/50-cloud-init.yaml.bak /etc/netplan/50-cloud-init.yaml sudo rm /etc/netplan/01-netcfg.yaml sudo netplan apply`

Problem: Netplan Try Hangs

Solution: Wait 120 seconds for automatic rollback, then check syntax:

```
sudo nano /etc/netplan/01-netcfg.yaml
# Verify YAML indentation (use spaces, not tabs!)
```

WinRM Connection Issues

Problem: Port 5985 Filtered/Closed

Diagnosis:

```
sudo nmap -p 5985 192.168.10.3
# Shows: filtered or closed
```

Solutions on Windows:

1. **Verify WinRM service:** `powershell Get-Service WinRM # Should show: Running`
2. **Check firewall rules:** `powershell Get-NetFirewallRule -DisplayName "*WinRM*" | Where-Object {$_.Enabled -eq $true}`
3. **Recreate firewall rule:** `powershell New-NetFirewallRule -DisplayName "WinRM HTTP In" -Direction Inbound -LocalPort 5985 -Protocol TCP -Action Allow`
4. **Verify WinRM listener:** `powershell winrm enumerate winrm/config/listener # Should show listener on port 5985`

Problem: Ansible win_ping Fails

Error: Authentication or permission failure

Solutions:

1. **Verify credentials in group_vars:** `bash sudo cat /etc/ansible/group_vars/windows.yml # Check username and password`
2. **Test WinRM manually:** `bash python3 << 'EOF' import winrm s = winrm.Session('192.168.10.3', auth=('username', 'password')) r = s.run_cmd('ipconfig') print(r.std_out.decode()) EOF`

Cloud-Init Issues

Problem: Hostname Reverts After Reboot

Solution: Ensure both configurations exist:

```
# Check preserve_hostname flag
sudo grep "preserve_hostname" /etc/cloud/cloud.cfg

# Check override file
cat /etc/cloud/cloud.cfg.d/99-disable-hostname.cfg
```

Problem: Network Config Overwritten

Solution: Verify network disable file:

```
cat /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
# Should contain: network: {config: disabled}
```

SSH Key Issues

Problem: SSH Key Not Working

Diagnosis:

```
# Check key permissions
ls -la ~/.ssh/
# Private key must be: -rw----- (600)
# Public key must be: -rw-r--r-- (644)
```

Fix permissions:

```
chmod 600 ~/.ssh/ansible-automation-key
chmod 644 ~/.ssh/ansible-automation-key.pub
chmod 600 ~/.ssh/config
```

Problem: SSH Still Uses Wrong Key

Solution: Verify SSH config:

```
cat ~/.ssh/config
# Must contain IdentityFile line

# Test which key SSH would use:
ssh -G hostname | grep identityfile
```

Security Considerations

Service Account Security

1. **Limited Scope:** svc-ansible used ONLY for automation
2. **No Interactive Login:** No direct SSH to svc-ansible from external hosts
3. **Strong Separation:** Different credentials from personal account
4. **Audit Trail:** Clear logs showing automation vs human actions

SSH Key Security

1. **No Passphrase:** Acceptable for automation in controlled environment
2. **File Permissions:** Strict 600 permissions on private key
3. **Access Control:** Key only accessible via svc-ansible account

4. **Custom Naming:** Clearly identifies purpose and usage

Network Security

1. **Static IP:** Predictable, easier to firewall
2. **Management VLAN:** Isolated from other networks
3. **pfSense Firewall:** Controls inter-VLAN traffic
4. **Limited Services:** Only SSH and necessary services exposed

Windows Security

1. **Basic Auth:** Acceptable for homelab, use HTTPS in production
2. **Local Admin:** Consider domain accounts in production
3. **Firewall Rules:** Explicitly allow only necessary ports
4. **Audit Logging:** Windows Event Logs track WinRM access

Best Practices Applied

Enterprise Patterns

1. **Service Accounts:** Dedicated accounts for automation
2. **Static Infrastructure:** Fixed IPs for core services
3. **Documentation:** Comprehensive setup documentation
4. **Configuration Management:** Ansible for consistent configuration
5. **Security Layers:** Multiple security controls

DevOps Principles

1. **Infrastructure as Code:** Netplan, Ansible configs in version control
2. **Idempotency:** Scripts can run multiple times safely
3. **Automation:** Repeatable processes via scripts
4. **Version Control:** All configurations documented and tracked

Operational Excellence

1. **Testing:** Netplan try provides safe rollback
2. **Backups:** Configuration backups before changes
3. **Verification:** Comprehensive post-change testing
4. **Monitoring:** Clear indicators of success/failure

Summary

This implementation transformed a basic Ubuntu VM into an enterprise-grade Ansible management server following professional best practices:

Achievements: -  Professional service account architecture -  Custom SSH key with descriptive naming
-  Static IP configuration with cloud-init protection -  Cross-platform management (Linux + Windows) -
 Comprehensive documentation and scripts -  Security-focused implementation

Key Principles: - Separation of duties (admin vs automation) - Infrastructure as code - Repeatability and automation - Comprehensive testing - Security by design

The server is now ready to manage enterprise infrastructure with the same patterns and practices used in production environments.

Implementation Status:  Complete and Operational

Related Documentation:

- [Ansible Service Account & Roles](#) - Service account conceptual architecture
- [Ansible Roles Architecture](#) - Role-based automation framework
- [Windows Integration](#) - Comprehensive Windows automation guide